

DL Lab 9

Name: Soumyadeep Ganguly

Reg No: 24MDT0082

```
In [22]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, TimeDistributed
from tensorflow.keras.callbacks import EarlyStopping

np.random.seed(42)
```

Question 1

```
In [3]: data = pd.read_csv("/content/gold_price.csv")
data = data['Price'].values.reshape(-1, 1)
print(data)
```

```
[[ 18.93]
 [ 18.93]
 [ 18.93]
 ...
 [3309.49]
 [3352.66]
 [3340.15]]
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
data_scaled = scaler.fit_transform(data)
```

```
print(data_scaled)
```

```
[[5.60618779e-04]
 [5.60618779e-04]
 [5.60618779e-04]
 ...
 [9.87057801e-01]
 [1.00000000e+00]
 [9.96249550e-01]]
```

```
In [5]: train_size = int(len(data_scaled)*0.8)
        train_data = data_scaled[:train_size]
        test_data = data_scaled[train_size:]
```

```
In [7]: def create_sequence(data, seq_length):
        sequences = []
        for i in range(len(data) - seq_length + 1):
            sequences.append(data[i: i+seq_length])
        return np.array(sequences)

        seq_length = 10
        train_sequences = create_sequence(train_data, seq_length)
        test_sequences = create_sequence(test_data, seq_length)
```

```
In [8]: X_train_full, y_train_full = train_sequences[:, :-1], train_sequences[:, -1]
        X_test, y_test = test_sequences[:, :-1], test_sequences[:, -1]
        val_fraction = 0.2
        val_size = int(len(X_train_full) * val_fraction)
        X_val = X_train_full[-val_size:]
        y_val = y_train_full[-val_size:]
        X_train = X_train_full[:-val_size]
        y_train = y_train_full[:-val_size]
```

```
In [9]: model = Sequential()
        model.add(SimpleRNN(64, activation='tanh', input_shape=(seq_length-1, 1)))
        model.add(Dense(1))
        model.compile(optimizer='adam', loss='mean_squared_error')
        model.summary()
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 64)	4,224
dense (Dense)	(None, 1)	65

Total params: 4,289 (16.75 KB)

Trainable params: 4,289 (16.75 KB)

Non-trainable params: 0 (0.00 B)

```
In [10]: e_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True, verbose = 1)
```

```
In [11]: hist = model.fit(
    X_train, y_train,
    epochs = 500,
    batch_size = 16,
    validation_data = (X_val, y_val),
    verbose = 1,
    callbacks = [e_stop]
)
```

Epoch 1/500
92/92  2s 8ms/step - loss: 3.6601e-05 - val_loss: 2.0329e-04
Epoch 2/500
92/92  0s 4ms/step - loss: 3.7479e-08 - val_loss: 1.9432e-04
Epoch 3/500
92/92  1s 7ms/step - loss: 3.3944e-08 - val_loss: 1.8751e-04
Epoch 4/500
92/92  1s 8ms/step - loss: 6.0600e-08 - val_loss: 1.7903e-04
Epoch 5/500
92/92  1s 7ms/step - loss: 3.9932e-08 - val_loss: 1.6985e-04
Epoch 6/500
92/92  1s 8ms/step - loss: 4.9256e-08 - val_loss: 1.6164e-04
Epoch 7/500
92/92  1s 6ms/step - loss: 5.1153e-08 - val_loss: 1.5668e-04
Epoch 8/500
92/92  0s 5ms/step - loss: 4.8846e-08 - val_loss: 1.4752e-04
Epoch 9/500
92/92  0s 4ms/step - loss: 3.5573e-08 - val_loss: 1.3675e-04
Epoch 10/500
92/92  0s 5ms/step - loss: 3.5047e-08 - val_loss: 1.2999e-04
Epoch 11/500
92/92  0s 5ms/step - loss: 3.3713e-08 - val_loss: 1.2372e-04
Epoch 12/500
92/92  0s 5ms/step - loss: 3.9193e-08 - val_loss: 1.2089e-04
Epoch 13/500
92/92  1s 5ms/step - loss: 3.9411e-08 - val_loss: 1.1104e-04
Epoch 14/500
92/92  0s 5ms/step - loss: 2.2549e-08 - val_loss: 1.0936e-04
Epoch 15/500
92/92  0s 5ms/step - loss: 3.3865e-08 - val_loss: 1.0243e-04
Epoch 16/500
92/92  0s 5ms/step - loss: 2.1093e-08 - val_loss: 9.7524e-05
Epoch 17/500
92/92  0s 5ms/step - loss: 3.2047e-08 - val_loss: 9.6466e-05
Epoch 18/500
92/92  0s 5ms/step - loss: 3.6150e-08 - val_loss: 9.1417e-05
Epoch 19/500
92/92  0s 5ms/step - loss: 3.2892e-08 - val_loss: 8.7732e-05
Epoch 20/500
92/92  0s 5ms/step - loss: 3.2892e-08 - val_loss: 8.7732e-05
Epoch 21/500
92/92  0s 5ms/step - loss: 4.4132e-08 - val_loss: 8.5607e-05

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

92/92  **0s** 5ms/step - loss: 3.1956e-08 - val_loss: 8.7648e-05
 Epoch 22/500
92/92  **0s** 4ms/step - loss: 5.3245e-08 - val_loss: 8.2960e-05
 Epoch 23/500
92/92  **0s** 5ms/step - loss: 2.5608e-08 - val_loss: 8.1609e-05
 Epoch 24/500
92/92  **0s** 5ms/step - loss: 3.0909e-08 - val_loss: 8.0110e-05
 Epoch 25/500
92/92  **1s** 6ms/step - loss: 2.9770e-08 - val_loss: 7.7386e-05
 Epoch 26/500
92/92  **0s** 5ms/step - loss: 1.9528e-08 - val_loss: 7.5697e-05
 Epoch 27/500
92/92  **0s** 5ms/step - loss: 2.4004e-08 - val_loss: 7.4302e-05
 Epoch 28/500
92/92  **1s** 6ms/step - loss: 2.7341e-08 - val_loss: 7.5415e-05
 Epoch 29/500
92/92  **1s** 8ms/step - loss: 1.8826e-08 - val_loss: 7.1993e-05
 Epoch 30/500
92/92  **1s** 8ms/step - loss: 2.8231e-08 - val_loss: 7.1506e-05
 Epoch 31/500
92/92  **1s** 5ms/step - loss: 3.1374e-08 - val_loss: 7.0460e-05
 Epoch 32/500
92/92  **0s** 4ms/step - loss: 3.5319e-08 - val_loss: 6.9257e-05
 Epoch 33/500
92/92  **0s** 5ms/step - loss: 2.2277e-08 - val_loss: 6.8327e-05
 Epoch 34/500
92/92  **0s** 4ms/step - loss: 4.3787e-08 - val_loss: 6.8532e-05
 Epoch 35/500
92/92  **0s** 4ms/step - loss: 1.5542e-08 - val_loss: 6.6255e-05
 Epoch 36/500
92/92  **0s** 5ms/step - loss: 1.7522e-08 - val_loss: 6.4993e-05
 Epoch 37/500
92/92  **0s** 5ms/step - loss: 3.4647e-08 - val_loss: 6.4060e-05
 Epoch 38/500
92/92  **0s** 5ms/step - loss: 2.4794e-08 - val_loss: 6.5156e-05
 Epoch 39/500
92/92  **0s** 4ms/step - loss: 2.1108e-08 - val_loss: 6.3100e-05
 Epoch 40/500
92/92  **0s** 5ms/step - loss: 1.6528e-08 - val_loss: 6.1606e-05
 Epoch 41/500
92/92  **0s** 5ms/step - loss: 3.5750e-08 - val_loss: 6.0408e-05
 Epoch 42/500

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

Epoch 42/500
92/92  1s 6ms/step - loss: 2.6163e-08 - val_loss: 5.9552e-05
Epoch 43/500
92/92  0s 4ms/step - loss: 4.1123e-08 - val_loss: 6.1344e-05
Epoch 44/500
92/92  0s 5ms/step - loss: 2.4302e-08 - val_loss: 5.7959e-05
Epoch 45/500
92/92  0s 5ms/step - loss: 2.2518e-08 - val_loss: 5.8094e-05
Epoch 46/500
92/92  0s 5ms/step - loss: 1.8900e-08 - val_loss: 5.6763e-05
Epoch 47/500
92/92  0s 5ms/step - loss: 2.3176e-08 - val_loss: 5.6467e-05
Epoch 48/500
92/92  0s 5ms/step - loss: 2.4439e-08 - val_loss: 5.5706e-05
Epoch 49/500
92/92  0s 5ms/step - loss: 5.3422e-08 - val_loss: 5.4612e-05
Epoch 50/500
92/92  0s 4ms/step - loss: 1.7879e-08 - val_loss: 5.4140e-05
Epoch 51/500
92/92  0s 5ms/step - loss: 5.3560e-08 - val_loss: 5.4065e-05
Epoch 52/500
92/92  1s 7ms/step - loss: 1.7477e-08 - val_loss: 5.9713e-05
Epoch 53/500
92/92  1s 7ms/step - loss: 9.3799e-08 - val_loss: 5.4111e-05
Epoch 54/500
92/92  1s 7ms/step - loss: 3.0604e-08 - val_loss: 5.2084e-05
Epoch 55/500
92/92  1s 5ms/step - loss: 1.6515e-08 - val_loss: 5.3262e-05
Epoch 56/500
92/92  0s 5ms/step - loss: 3.5159e-08 - val_loss: 5.0127e-05
Epoch 57/500
92/92  0s 4ms/step - loss: 2.8513e-08 - val_loss: 5.0728e-05
Epoch 58/500
92/92  0s 4ms/step - loss: 6.7531e-08 - val_loss: 5.4150e-05
Epoch 59/500
92/92  0s 5ms/step - loss: 2.4162e-08 - val_loss: 4.9566e-05
Epoch 60/500
92/92  0s 5ms/step - loss: 3.0533e-08 - val_loss: 4.8859e-05
Epoch 61/500
92/92  0s 5ms/step - loss: 3.0533e-08 - val_loss: 4.8859e-05
Epoch 62/500
92/92  0s 5ms/step - loss: 4.6175e-08 - val_loss: 5.3290e-05

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

92/92  **0s** 5ms/step - loss: 3.7024e-08 - val_loss: 4.6691e-05
 Epoch 63/500
92/92  **0s** 5ms/step - loss: 2.7825e-08 - val_loss: 4.6291e-05
 Epoch 64/500
92/92  **0s** 5ms/step - loss: 3.2027e-08 - val_loss: 4.8093e-05
 Epoch 65/500
92/92  **0s** 5ms/step - loss: 2.3014e-08 - val_loss: 4.5867e-05
 Epoch 66/500
92/92  **0s** 5ms/step - loss: 4.4938e-08 - val_loss: 4.5678e-05
 Epoch 67/500
92/92  **0s** 5ms/step - loss: 2.5229e-08 - val_loss: 4.4152e-05
 Epoch 68/500
92/92  **0s** 5ms/step - loss: 4.6633e-08 - val_loss: 4.4264e-05
 Epoch 69/500
92/92  **1s** 5ms/step - loss: 4.4931e-08 - val_loss: 4.6150e-05
 Epoch 70/500
92/92  **0s** 5ms/step - loss: 3.4189e-08 - val_loss: 4.5120e-05
 Epoch 71/500
92/92  **0s** 5ms/step - loss: 1.7603e-08 - val_loss: 4.3064e-05
 Epoch 72/500
92/92  **0s** 5ms/step - loss: 4.0195e-08 - val_loss: 4.6507e-05
 Epoch 73/500
92/92  **0s** 5ms/step - loss: 1.0348e-07 - val_loss: 4.3777e-05
 Epoch 74/500
92/92  **0s** 5ms/step - loss: 2.2015e-08 - val_loss: 4.1659e-05
 Epoch 75/500
92/92  **1s** 7ms/step - loss: 1.4073e-08 - val_loss: 4.2583e-05
 Epoch 76/500
92/92  **1s** 7ms/step - loss: 2.9464e-08 - val_loss: 4.0775e-05
 Epoch 77/500
92/92  **1s** 8ms/step - loss: 1.3807e-08 - val_loss: 4.9956e-05
 Epoch 78/500
92/92  **1s** 7ms/step - loss: 4.0132e-08 - val_loss: 4.2160e-05
 Epoch 79/500
92/92  **0s** 5ms/step - loss: 1.9651e-08 - val_loss: 4.0243e-05
 Epoch 80/500
92/92  **0s** 4ms/step - loss: 3.8144e-08 - val_loss: 4.2674e-05
 Epoch 81/500
92/92  **0s** 4ms/step - loss: 4.4981e-08 - val_loss: 4.0627e-05
92/92  **0s** 5ms/step - loss: 3.3848e-08 - val_loss: 3.9195e-05

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```

Epoch 83/500
92/92  0s 5ms/step - loss: 3.5310e-08 - val_loss: 3.8949e-05
Epoch 84/500
92/92  0s 5ms/step - loss: 4.1160e-08 - val_loss: 3.8751e-05
Epoch 85/500
92/92  0s 5ms/step - loss: 3.0270e-08 - val_loss: 3.8649e-05
Epoch 86/500
92/92  0s 5ms/step - loss: 2.7763e-08 - val_loss: 3.8050e-05
Epoch 87/500
92/92  0s 4ms/step - loss: 3.8538e-08 - val_loss: 4.0018e-05
Epoch 88/500
92/92  0s 4ms/step - loss: 4.5252e-08 - val_loss: 4.0756e-05
Epoch 89/500
92/92  0s 5ms/step - loss: 2.2609e-08 - val_loss: 3.8654e-05
Epoch 90/500
92/92  1s 4ms/step - loss: 4.1632e-08 - val_loss: 3.7798e-05
Epoch 91/500
92/92  1s 8ms/step - loss: 5.8700e-08 - val_loss: 3.7506e-05
Epoch 92/500
92/92  1s 7ms/step - loss: 2.5018e-08 - val_loss: 3.8466e-05
Epoch 93/500
92/92  0s 4ms/step - loss: 2.6747e-08 - val_loss: 3.7281e-05
Epoch 94/500
92/92  1s 7ms/step - loss: 5.3992e-08 - val_loss: 3.7294e-05
Epoch 95/500
92/92  0s 4ms/step - loss: 3.6730e-08 - val_loss: 3.7250e-05
Epoch 96/500
92/92  0s 5ms/step - loss: 3.4033e-08 - val_loss: 3.6452e-05
Epoch 97/500
92/92  0s 5ms/step - loss: 2.6208e-08 - val_loss: 3.6234e-05
Epoch 98/500
92/92  0s 5ms/step - loss: 2.5474e-08 - val_loss: 3.6832e-05
Epoch 99/500
92/92  1s 8ms/step - loss: 1.3100e-08 - val_loss: 3.6419e-05
Epoch 100/500
92/92  1s 8ms/step - loss: 3.1650e-08 - val_loss: 4.7236e-05
Epoch 101/500
92/92  1s 4ms/step - loss: 2.7012e-08 - val_loss: 4.2998e-05
Epoch 102/500
Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js ms/step - loss: 7.8718e-08 - val_loss: 4.6565e-05

```

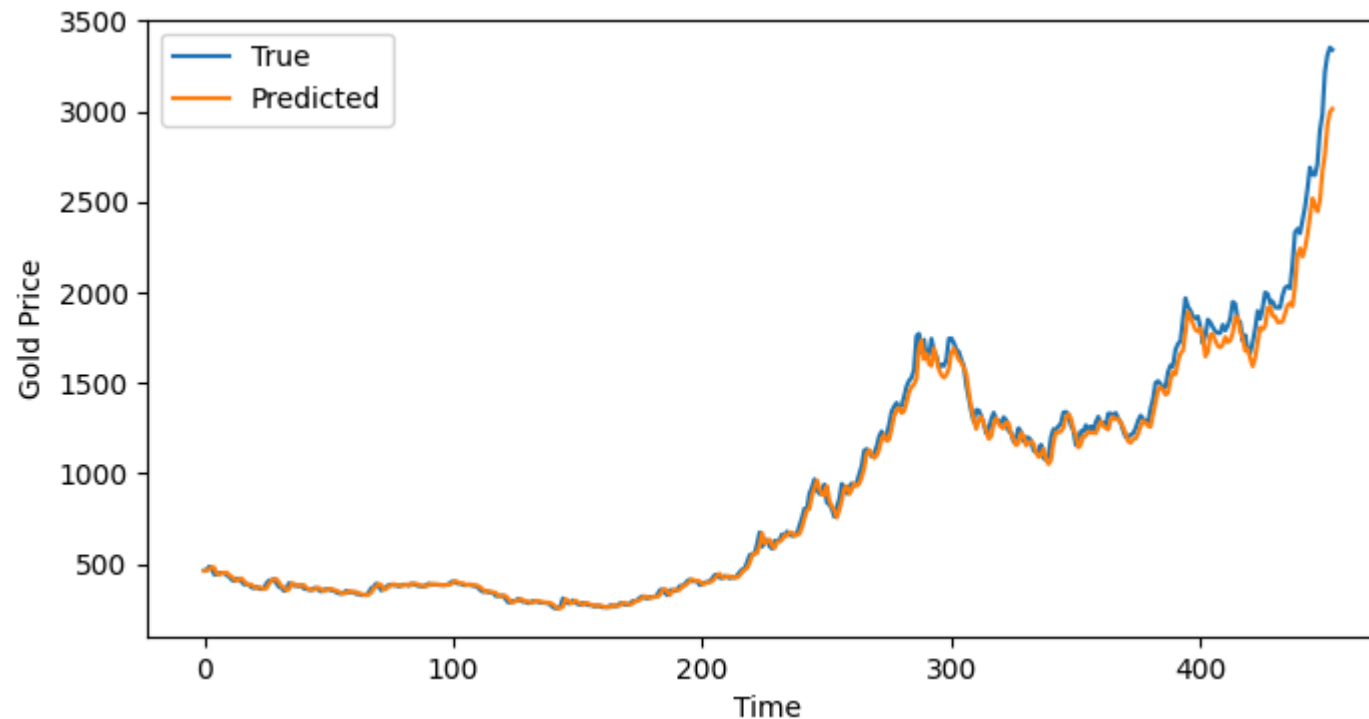
Epoch 102: early stopping

Restoring model weights from the end of the best epoch: 97.

```
In [12]: y_pred = model.predict(X_test)
y_test_orig = scaler.inverse_transform(y_test)
y_pred_orig = scaler.inverse_transform(y_pred)
rootmse = np.sqrt(mean_squared_error(y_test_orig, y_pred_orig))
print(f"Root mean Squared Error: {rootmse}")
plt.figure(figsize=(8,4))
plt.plot(y_test_orig, label="True")
plt.plot(y_pred_orig, label="Predicted")
plt.xlabel('Time')
plt.ylabel('Gold Price')
plt.legend()
plt.show()
```

15/15 ————— 1s 37ms/step

Root mean Squared Error: 72.26223471534979



Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

Question 2

```
In [13]: data = pd.read_csv("gold_price.csv")
data = data['Price'].values.reshape(-1, 1)
print(data)
```

```
[[ 18.93]
 [ 18.93]
 [ 18.93]
 ...
 [3309.49]
 [3352.66]
 [3340.15]]
```

```
In [14]: scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
```

```
In [15]: print(data_scaled)
```

```
[[5.60618779e-04]
 [5.60618779e-04]
 [5.60618779e-04]
 ...
 [9.87057801e-01]
 [1.00000000e+00]
 [9.96249550e-01]]
```

```
In [16]: train_size = int(len(data_scaled)*0.8)
train_data = data_scaled[:train_size]
test_data = data_scaled[train_size:]
```

```
In [19]: def create_sequence_len(data, input_len = 10, output_len = 10):
X, y = [], []
for i in range(len(data) - input_len - output_len + 1):
    X.append(data[i:i+input_len])
    y.append(data[i+input_len: i+input_len + output_len])
return np.array(X), np.array(y)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
input_len = 10
output_len = 10
```

```
In [26]: model2 = Sequential()
model2.add(SimpleRNN(64, activation='tanh', return_sequences=True, input_shape=(input_len, 1)))
model2.add(TimeDistributed(Dense(1)))
model2.compile(optimizer = "adam", loss = "mean_squared_error")
model2.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
simple_rnn_5 (SimpleRNN)	(None, 10, 64)	4,224
time_distributed_2 (TimeDistributed)	(None, 10, 1)	65

Total params: 4,289 (16.75 KB)

Trainable params: 4,289 (16.75 KB)

Non-trainable params: 0 (0.00 B)

```
In [30]: X_train_full, y_train_full = create_sequence_len(train_data, input_len, output_len)
X_test, y_test = create_sequence_len(test_data, input_len, output_len)
val_fraction = 0.2
val_size = int(len(X_train_full) * val_fraction)
X_val = X_train_full[-val_size:]
y_val = y_train_full[-val_size:]
X_train = X_train_full[:-val_size]
y_train = y_train_full[:-val_size]
```

```
In [25]: e_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True, verbose = 1)
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
In [31]: hist2 = model2.fit(
    X_train, y_train,
    epochs = 100,
    batch_size = 16,
    validation_data = (X_val, y_val),
    verbose = 1,
    callbacks = [e_stop]
)
```

```
Epoch 1/100
92/92 ————— 1s 6ms/step - loss: 2.2329e-07 - val_loss: 4.2806e-04
Epoch 2/100
92/92 ————— 1s 6ms/step - loss: 1.6220e-07 - val_loss: 4.2880e-04
Epoch 3/100
92/92 ————— 1s 5ms/step - loss: 1.7070e-07 - val_loss: 4.2051e-04
Epoch 4/100
92/92 ————— 1s 6ms/step - loss: 1.9995e-07 - val_loss: 4.2014e-04
Epoch 5/100
92/92 ————— 1s 5ms/step - loss: 1.5012e-07 - val_loss: 4.2111e-04
Epoch 6/100
92/92 ————— 1s 6ms/step - loss: 2.3036e-07 - val_loss: 4.2472e-04
Epoch 7/100
92/92 ————— 1s 5ms/step - loss: 1.4245e-07 - val_loss: 4.2012e-04
Epoch 8/100
92/92 ————— 1s 5ms/step - loss: 1.5169e-07 - val_loss: 4.2082e-04
Epoch 9/100
92/92 ————— 1s 5ms/step - loss: 1.3090e-07 - val_loss: 4.2198e-04
Epoch 10/100
92/92 ————— 1s 5ms/step - loss: 1.2366e-07 - val_loss: 4.2731e-04
Epoch 11/100
92/92 ————— 1s 5ms/step - loss: 1.4409e-07 - val_loss: 4.2353e-04
Epoch 12/100
92/92 ————— 1s 5ms/step - loss: 1.2700e-07 - val_loss: 4.2199e-04
Epoch 12: early stopping
Restoring model weights from the end of the best epoch: 7.
```

```
In [33]: y_pred2 = model2.predict(X_test)
```

```
y_test_orig2 = scaler.inverse_transform(y_test.reshape(-1, 1))
transform(y_pred2.reshape(-1, 1))
```

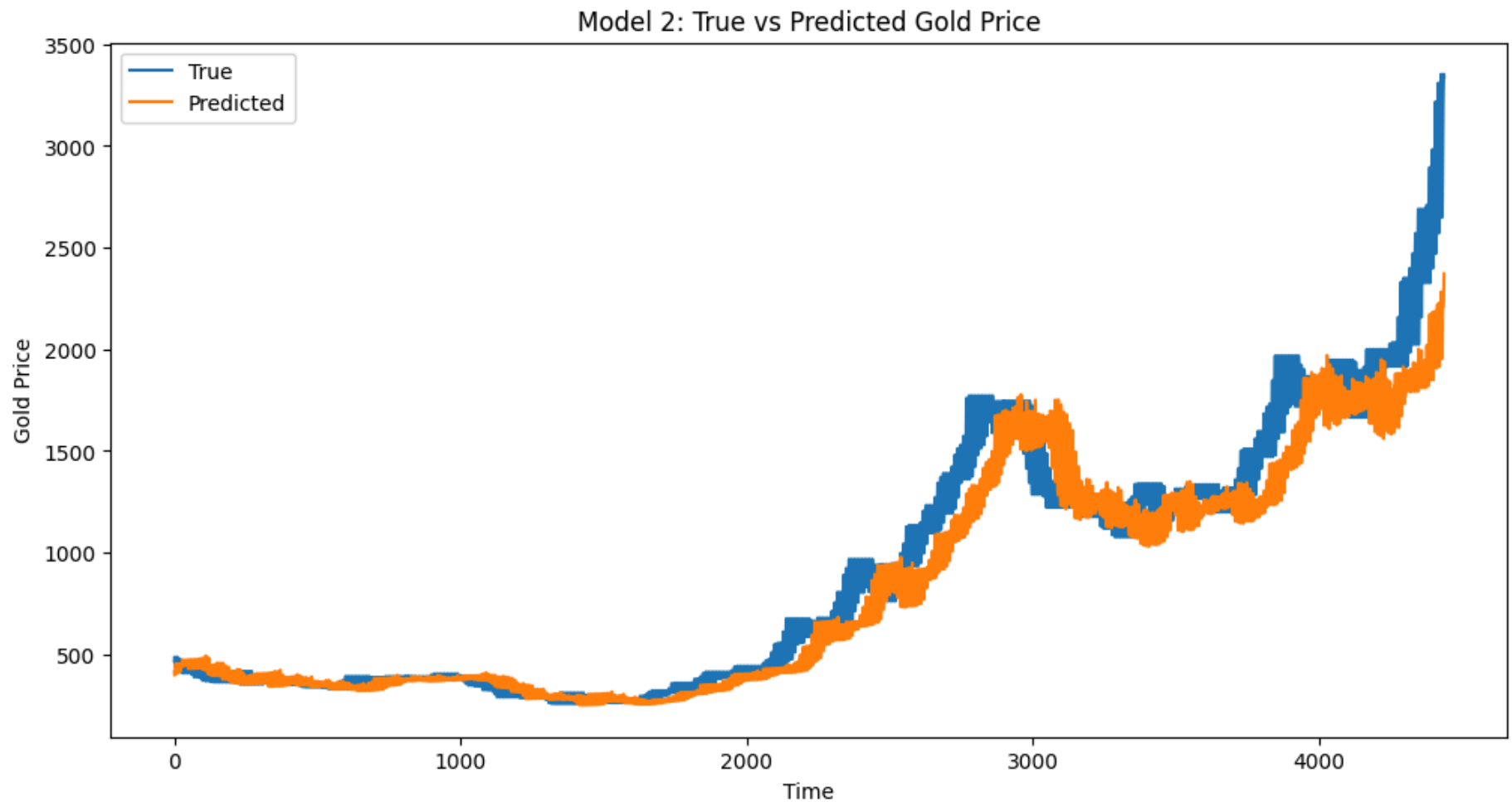
Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
rootmse2 = np.sqrt(mean_squared_error(y_test_orig2, y_pred_orig2))
print(f"Root mean Squared Error for model2: {rootmse2}")

plt.figure(figsize=(12, 6))
plt.plot(y_test_orig2[:, 0], label="True")
plt.plot(y_pred_orig2[:, 0], label="Predicted")
plt.xlabel('Time')
plt.ylabel('Gold Price')
plt.title('Model 2: True vs Predicted Gold Price')
plt.legend()
plt.show()
```

14/14 ————— 0s 5ms/step

Root mean Squared Error for model2: 179.5156235298048



Question 3

```
In [34]: model3 = Sequential()  
model3.add(SimpleRNN(128, activation='tanh', return_sequences=True, input_shape=(input_len, 1)))  
model3.add(SimpleRNN(64, activation='tanh', return_sequences=True))  
model3.add(SimpleRNN(32, activation='tanh', return_sequences=True))  
model3.add(Dense(1))
```

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
model3.compile(optimizer='adam', loss='mean_squared_error')
model3.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
simple_rnn_6 (SimpleRNN)	(None, 10, 128)	16,640
simple_rnn_7 (SimpleRNN)	(None, 10, 64)	12,352
simple_rnn_8 (SimpleRNN)	(None, 10, 32)	3,104
dense_4 (Dense)	(None, 10, 1)	33

Total params: 32,129 (125.50 KB)

Trainable params: 32,129 (125.50 KB)

Non-trainable params: 0 (0.00 B)

```
In [35]: hist3 = model3.fit(
    X_train, y_train,
    epochs = 100,
    batch_size = 16,
    validation_data = (X_val, y_val),
    verbose = 1,
    callbacks = [e_stop]
)
```

Epoch 1/100
92/92  5s 16ms/step - loss: 8.2065e-04 - val_loss: 0.0015
Epoch 2/100
92/92  1s 10ms/step - loss: 1.1246e-06 - val_loss: 9.0088e-04
Epoch 3/100
92/92  1s 10ms/step - loss: 4.5771e-07 - val_loss: 6.8716e-04
Epoch 4/100
92/92  1s 10ms/step - loss: 3.2278e-07 - val_loss: 5.9312e-04
Epoch 5/100
92/92  1s 10ms/step - loss: 2.2116e-07 - val_loss: 5.4587e-04
Epoch 6/100
92/92  1s 15ms/step - loss: 2.1359e-07 - val_loss: 5.1904e-04
Epoch 7/100
92/92  2s 19ms/step - loss: 2.0819e-07 - val_loss: 5.0089e-04
Epoch 8/100
92/92  1s 10ms/step - loss: 1.6959e-07 - val_loss: 4.8747e-04
Epoch 9/100
92/92  1s 10ms/step - loss: 1.7968e-07 - val_loss: 4.7750e-04
Epoch 10/100
92/92  1s 10ms/step - loss: 1.7200e-07 - val_loss: 4.6639e-04
Epoch 11/100
92/92  1s 10ms/step - loss: 1.6832e-07 - val_loss: 4.6171e-04
Epoch 12/100
92/92  1s 10ms/step - loss: 1.1894e-07 - val_loss: 4.5379e-04
Epoch 13/100
92/92  1s 11ms/step - loss: 1.3872e-07 - val_loss: 4.4587e-04
Epoch 14/100
92/92  1s 10ms/step - loss: 2.1585e-07 - val_loss: 4.5001e-04
Epoch 15/100
92/92  1s 10ms/step - loss: 1.5944e-07 - val_loss: 4.4548e-04
Epoch 16/100
92/92  1s 11ms/step - loss: 1.1130e-07 - val_loss: 4.4222e-04
Epoch 17/100
92/92  1s 12ms/step - loss: 1.1998e-07 - val_loss: 4.4231e-04
Epoch 18/100
92/92  2s 18ms/step - loss: 1.0152e-07 - val_loss: 4.4011e-04
Epoch 19/100
92/92  2s 11ms/step - loss: 1.2149e-07 - val_loss: 4.3495e-04
Epoch 20/100
Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js 0ms/step - loss: 1.0734e-07 - val_loss: 4.3741e-04
Epoch 21/100

```

92/92 ————— 1s 10ms/step - loss: 1.4260e-07 - val_loss: 4.3771e-04
Epoch 22/100
92/92 ————— 1s 10ms/step - loss: 1.0877e-07 - val_loss: 4.3733e-04
Epoch 23/100
92/92 ————— 1s 11ms/step - loss: 1.7712e-07 - val_loss: 4.3391e-04
Epoch 24/100
92/92 ————— 1s 10ms/step - loss: 1.3641e-07 - val_loss: 4.3568e-04
Epoch 25/100
92/92 ————— 1s 10ms/step - loss: 1.4246e-07 - val_loss: 4.3646e-04
Epoch 26/100
92/92 ————— 1s 10ms/step - loss: 1.5877e-07 - val_loss: 4.3850e-04
Epoch 27/100
92/92 ————— 1s 10ms/step - loss: 1.9167e-07 - val_loss: 4.3604e-04
Epoch 28/100
92/92 ————— 2s 16ms/step - loss: 2.0776e-07 - val_loss: 4.3583e-04
Epoch 28: early stopping
Restoring model weights from the end of the best epoch: 23.

```

```

In [36]: y_pred3 = model3.predict(X_test)

y_test_orig3 = scaler.inverse_transform(y_test.reshape(-1, 1))
y_pred_orig3 = scaler.inverse_transform(y_pred2.reshape(-1, 1))

rootmse3 = np.sqrt(mean_squared_error(y_test_orig3, y_pred_orig3))
print(f"Root mean Squared Error for model3: {rootmse3}")

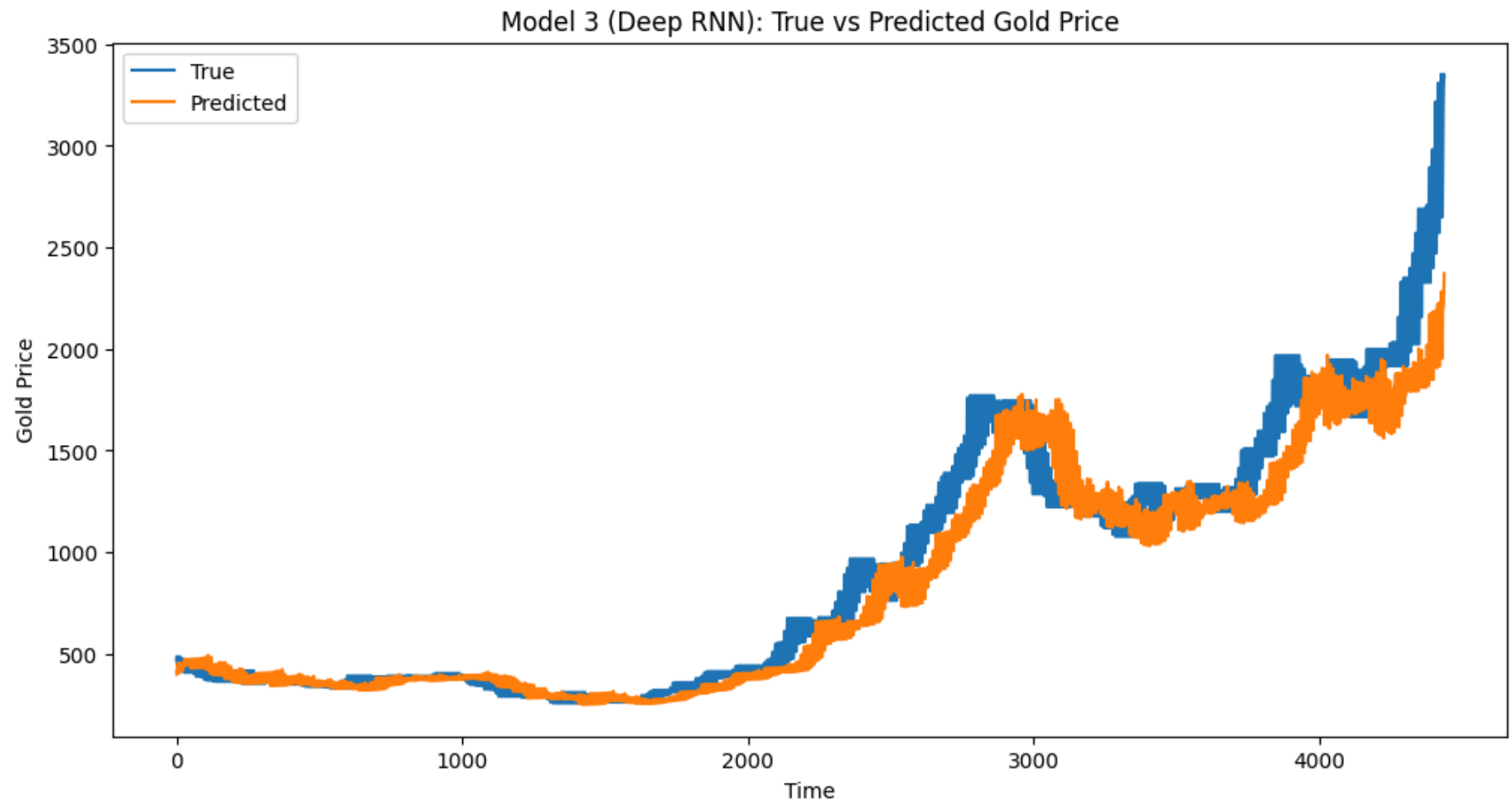
plt.figure(figsize=(12, 6))
plt.plot(y_test_orig2[:, 0], label="True")
plt.plot(y_pred_orig2[:, 0], label="Predicted")
plt.xlabel('Time')
plt.ylabel('Gold Price')
plt.title('Model 3 (Deep RNN): True vs Predicted Gold Price')
plt.legend()
plt.show()

```

```

14/14 ————— 1s 41ms/step
Root mean Squared Error for model3: 179.5156235298048

```



In []: